MediDrone Istanbul

Comparing a Genetic Algorithm and Tabu Search
for Capacitated Medical Delivery

Faruk Alpay (2267273)
CMP3005 Analysis of Algorithms, Fall 2026–2027
Bahçeşehir University
Instructor: Dr. Günet Eroğlu

Abstract

This project compares two ways to plan medical deliveries with capacity-limited drones: a Genetic Algorithm (GA) and Tabu Search (TS). A Clarke–Wright savings heuristic provides a fast starting solution. The main evaluation uses the four required CVRPLIB benchmarks and a generated instance with 100 clinics. Each stochastic method has ten recorded runs and the same time budget per instance. In these saved experiments, TS gives lower mean costs than the pure GA on every main instance. Both methods reach the exact optimum in all runs on nine tiny test cases. The report explains the required reductions, Split dynamic program and running-time analysis using small examples. All tables and plots come from existing experiment files.

1 Problem and project scope

A central laboratory must send medical supplies to clinics. A delivery plan answers two questions: *which clinics belong to each drone, and in which order should that drone visit them?* Every drone starts and finishes at the laboratory. Crossing the Bosphorus adds a fixed cost.

Inputs and a valid plan

Travel costs form a complete undirected graph. Demands are positive integers at most Q. There are n clinics and a depot numbered 0. Clinic i needs q_i units. Each drone can carry at most Q units, and at most K drones are available. The travel cost between i and j is $d(i, j)$. A route such as $(0, 3, 1, 0)$ serves clinics 3 and 1 and has cost $d(0, 3) + d(3, 1) + d(1, 0)$.

A valid solution visits every clinic exactly once, uses at most K nonempty routes, and keeps the sum of demands on each route at most Q. For example, with $Q = 10$, demands 4 and 5 can share a route because $4 + 5 \leq 10$. Adding a demand of 3 makes that route invalid.

Optimization version: find a valid plan with the smallest total route cost. **Decision version:** given a bound B, does a valid plan with total cost at most B exist? The decision version supports the NP-completeness argument on the next page.

What this report evaluates

The core comparison is Clarke–Wright, GA and TS. The code also contains a memetic GA, real Istanbul data and wider benchmarks. These extensions remain available, but are outside the main comparison here. The required synthetic instance uses seed 7273, obtained from student ID 2267273.

Key point. The project combines assigning clinics to drones with ordering the visits. A short route is useful only if it also satisfies the capacity and fleet limits.

2 Why the decision problem is NP-complete

Checking a proposed solution: membership in NP

A certificate is the list of routes. Create a Boolean array with one entry per clinic. Scan the routes, mark their clinics, reject duplicate or invalid IDs, add demands and edge costs. Finally check that every clinic was seen, each load is at most Q, at most K routes are used, and the cost is at most B.

A valid certificate contains n clinic visits and at most $n+K \leq 2n$ edges, taking $K \leq n$. With a distance matrix already available, verification takes $O(n)$ arithmetic operations and $O(n)$ extra memory. Thus CVRP decision is in NP. The formal argument uses finitely encoded rational or integer edge costs. Building the matrix from coordinates is separate $O(n^2)$ preprocessing; the program uses a tolerance for floating-point cost checks.

Step A: Hamiltonian Cycle to TSP decision

Start with an undirected graph H with N vertices. Hamiltonian Cycle asks whether there is a cycle visiting every vertex exactly once [1]. Construct a complete graph on the same vertices. Give an edge cost 1 if it exists in H and cost 2 otherwise. Set the TSP bound to $B = N$. Constructing these costs examines $O(N^2)$ pairs.

Forward direction. A Hamiltonian cycle of H uses N original edges. Each costs 1 in the new graph, giving a TSP tour of cost N.

Reverse direction. Every TSP tour has N edges costing at least 1. A total cost at most N forces every edge to cost 1. Those edges all belong to H, giving a Hamiltonian cycle.

The construction is metric: a direct edge costs at most 2, while a detour through a distinct third vertex costs at least $1 + 1 = 2$.

Step B: TSP decision to CVRP decision

Take a TSP instance with $n + 1$ vertices. Choose one as depot 0; the other n become clinics. Copy the distances, set every demand to 1, set $Q = n$, set $K = 1$, and keep bound B. Copying the matrix takes $O(n^2)$ time.

A TSP tour becomes a single CVRP route with load n, so it is valid and has the same cost. Conversely, with one vehicle, a valid CVRP solution is a single depot-to-depot tour visiting all clinics. It gives a TSP tour of the same cost. Both directions preserve the answer.

Why the chain works

The chain is Hamiltonian Cycle $\leq_p$ TSP decision $\leq_p$ CVRP decision. A polynomial-time construction cannot produce more than polynomially many symbols. Applying another polynomial-time construction to that output still takes polynomial time. Since Hamiltonian Cycle is NP-complete, CVRP decision is NP-hard. Together with membership in NP, it is NP-complete.

Key point. An exact polynomial-time algorithm for every CVRP instance would also solve the known-hard problem at the start of the chain. This motivates searching for good feasible plans within a time budget. NP-hardness does not prove that every particular instance is difficult.

3 Capacity feasibility and the crossing penalty

Bin Packing inside the routing problem

Bin Packing decision gives item sizes $a_1, \ldots, a_n$, bin capacity C and at most k bins. Turn each item into a clinic with demand $q_i = a_i$, and set $Q = C$ and $K = k$. Use distance 1 between distinct locations, since distance is irrelevant to this feasibility question. If an item exceeds C, the input is immediately a NO instance.

A packing gives one route per bin, visiting its items in any order. Loads are at most Q and there are at most K routes. Conversely, a feasible route assignment gives bins containing those same demands. Both directions preserve the answer, and the construction is polynomial. A route assignment is also easy to check. Thus even the distance-free fleet-feasibility problem is NP-complete, by reduction from Bin Packing [1].

For example, demands $(6, 6, 6)$ total 18. Two drones of capacity 10 have total capacity 20, but cannot serve these demands: no two fit together. Total capacity alone does not guarantee feasibility.

This matters in the code. CW respects individual capacities but may produce too many routes. GA checks decoded route counts as well as loads. TS may temporarily overload routes, but saves a final answer only when every constraint holds.

Why a fixed crossing cost preserves the metric

Let $e(i, j)$ be unrounded Euclidean distance and $c(i, j)$ be 1 when the endpoints are on different sides of the Bosphorus, and 0 otherwise. Define

$$d(i, j) = e(i, j) + P\, c(i, j), \qquad P \geq 0.$$

Symmetry and zero self-distance are preserved. For the triangle inequality, consider going from i to k via j. If i and k are on different sides, j must be on one side, so at least one leg of the detour also crosses. If they are on the same side, their direct crossing cost is zero. In either case,

$$c(i, k) \leq c(i, j) + c(j, k).$$

Adding this inequality, multiplied by P, to the Euclidean triangle inequality gives $d(i, k) \leq d(i, j) + d(j, k)$. The crossing penalty therefore preserves the metric property.

The synthetic boundary is $x = 500$. The depot at $(500, 500)$ belongs to Asia because the rule is $x \geq 500$. A route serving Europe must cross and return.

Benchmark rounding is a separate convention

CVRPLIB requires Euclidean distances rounded to the nearest integer for comparison with its reference costs. Rounding can introduce small triangle-inequality violations. The exact proof above applies to unrounded MediDrone costs. The benchmark evaluator follows the benchmark's integer convention.

Key point. A plan must pass both capacity and fleet checks. The crossing proof uses a simple fact: a direct trip cannot require more side changes than a detour.

4 Data and experimental setup

Public benchmarks. A-n32-k5, A-n45-k6, A-n60-k9 and A-n80-k10 contain 31, 44, 59 and 79 customers; the name also counts the depot. Their reference costs are 784, 944, 1354 and 1763. Files came from the cached Hugging Face mirror of CVRPLIB [5,6]. The project's TSPLIB/CVRPLIB parser uses nearest-integer distances.

Team instance. Student ID 2267273 gives seed 7273. The generator places the depot at $(500, 500)$ and creates 100 customers in clusters, with about 20% uniform points. Demands are integers from 1 to 30, capacity is $Q = 100$, and $K = \lceil \sum_i q_i/100 \rceil + 1 = 18$. The boundary is $x = 500$; each crossing costs $P = 150$. These are abstract coordinate/cost units, not real kilometres.

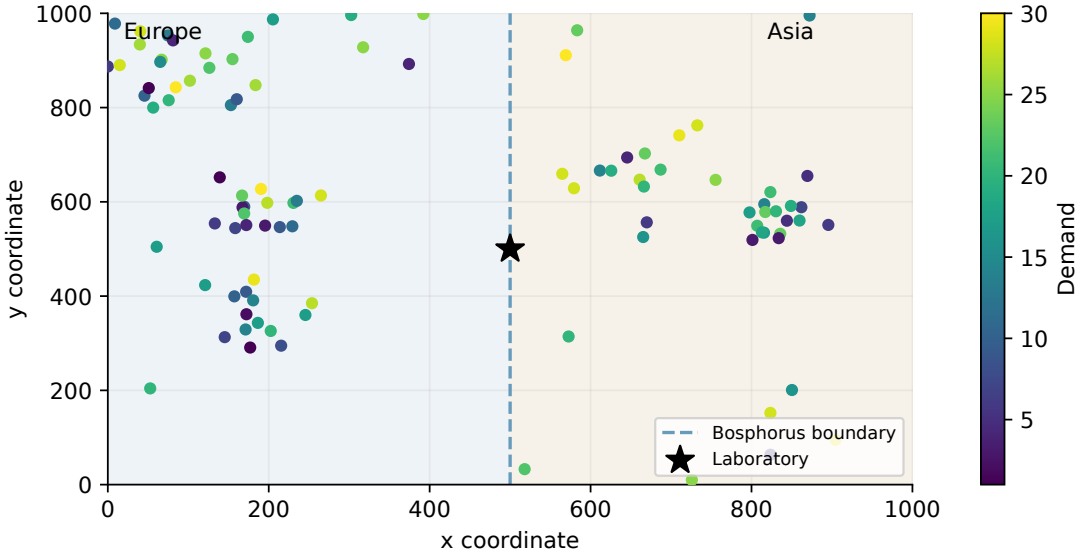


Figure 1: The required synthetic instance. Colour indicates demand; the star is the depot.

Tiny cases. Nine stored instances have 7, 8 or 9 customers, three of each, with $Q = 50$. Existing exact values agree between dynamic programming and MILP. This revision reads those cached values without solving the instances again.

Protocol. Evaluation seeds are 1–10. GA and TS each get 30 seconds on the two smaller benchmarks and 60 seconds on the others and the team instance. Tiny runs get 1 second. CW runs once. Tuning uses separate seeds 101–105 and 10 seconds per run.

Recorded hardware. Apple M4 Pro, 14 cores, 48 GB RAM, macOS 26.3, Apple clang 17 with `-O3 -march=native`. Solver processes are single-threaded. Main experiments used eight concurrent processes; the timing study ran sequentially. Shared hardware can introduce timing noise.

Final settings. GA: population 100, crossover 0.9, mutation 1.0, restart after 400 idle generations. TS: candidate list 20, tenure 10–25, penalty multiplier 1.3, restart after $\max(2000, 20n)$ idle iterations. Mutation 1.0 means one random mutation per child, not a change to every position. All settings and seeds remain in the saved JSON results.

5 Constructive baseline and Genetic Algorithm

Clarke–Wright savings

The savings method [2] starts with one drone per clinic. Joining routes removes two depot connections and adds an edge between the clinics. The saving is

$$s(i, j) = d(0, i) + d(0, j) - d(i, j).$$

If the depot legs cost 10 and 12 and the clinic-to-clinic edge costs 5, merging saves 17. Try larger savings first. Merge only suitable endpoints of distinct routes whose combined load fits.

Algorithm 1 Clarke–Wright baseline (`savings.cpp`)

Create a route $(0, i, 0)$ for every clinic i
Compute pair savings and sort in decreasing order
for each pair (i, j) in that order **do**
    if i, j are suitable endpoints of distinct routes and combined load is $\leq Q$ **then**
        Orient and merge those routes
Return routes and check their number against K

CW has no backtracking. Its routes can each fit capacity while exceeding the fleet limit. This happens on A-n45-k6 and the team instance. Such outputs are labelled infeasible.

Genetic Algorithm: many candidate orders

A chromosome is a permutation such as $(3, 1, 4, 2)$ without drone boundaries. Split converts it into routes. Fitness favours fewer excess routes first, then lower cost. Only fleet-feasible solutions can become the reported best answer.

Algorithm 2 Steady-state GA (`ga.cpp`)

Initialise population with the CW order and random permutations; decode them
while time remains **do**
    for each of μ offspring in a generation **do**
        Choose parents by tournaments between two random individuals
        Apply Order Crossover with probability p_c
        Apply swap, inversion or insertion with probability p_m
        Decode with Split; reject a duplicate decoded solution
        Save the child if it improves the best feasible answer
        Replace a random member of the worse half; restore population order
    Partially restart after stagnation, keeping the best fraction
Return the best feasible solution

Order Crossover [9] copies a segment from one parent, then fills remaining positions in the other parent's order, skipping used clinics. It preserves a valid permutation. Mutations change that order. Duplicate rejection and restarts help retain variety; replacement protects the best individual. This main GA uses no local search. The optional memetic variant does.

6 Split: turning an order into routes

A small example

Take giant tour $(1, 2, 3)$, demands $(4, 3, 4)$ and $Q = 7$. All clinics cannot share a drone. Possible cuts include $(1, 2) \mid (3)$ and $(1) \mid (2, 3)$. Split chooses the cheapest valid cut for this fixed order. It does not search every permutation.

Put the depot at position 0 on a line and clinics 1, 2 and 3 at positions 1, 2 and 3. Use ordinary distances without a crossing penalty. The first cut costs $4 + 6 = 10$; the second costs $2 + 6 = 8$. Split should return the second cut.

The directed acyclic graph

For tour $(t_1, \ldots, t_n)$, create nodes $0, 1, \ldots, n$. Node j means that the first j clinics have been served. Arc (i, j) exists if the consecutive block $t_{i+1}, \ldots, t_j$ fits one drone. Its cost is that block's depot-to-depot route cost:

$$C(i, j) = d(0, t_{i+1}) + \sum_{h=i+1}^{j-1} d(t_h, t_{h+1}) + d(t_j, 0).$$

Every arc goes forward, so there are no directed cycles. A path from 0 to n is a sequence of cuts covering the whole tour.

Dynamic-programming recurrence

Let $D[j]$ be the cheapest way to serve the first j clinics. Set $D[0] = 0$. Then

$$D[j] = \min_{\substack{0 \le i < j \\ \sum_{h=i+1}^{j} q_{t_h} \le Q}} (D[i] + C(i, j)).$$

Store the minimizing i to reconstruct routes backwards from n. In the example, $D[1] = 2$, $D[2] = 4$ and $D[3] = \min\{D[1] + 6, D[2] + 6\} = 8$.

Correctness. The last route in an optimal solution is some feasible consecutive block ending at j. Its earlier routes must be optimal for their prefix: otherwise replacing them by cheaper routes would improve the whole answer. The recurrence checks every last block and combines it with the optimal prefix.

Running time and the fleet limit

There are n end positions and at most n starts each: $O(n^2)$ time. If a route can contain at most L consecutive clinics, only L starts need checking, giving $O(nL)$. Prefix sums provide constant-time block-load and block-cost queries.

This basic recurrence does not limit route count. A state $D[r, j]$ for exactly r routes, minimised over $r \le K$, enforces the fleet. Direct layered DP costs $O(KnL)$. The existing fast decoder [3,4] reuses candidate starts in a queue, giving $O(n)$ for unlimited routes and $O(Kn)$ for the layered version. The ordinary DP remains as `split_bellman` for explanation and reference.

In `ga.cpp`, fast unlimited Split runs first. If it uses too many routes but the order can fit within K, fleet-limited Split runs. Otherwise the chromosome is marked fleet-infeasible.

Key point. Split is exact for the given order. An optimal cut of a poor order can still give a poor plan.

7 Tabu Search: improving one route plan

TS stores a route plan directly. It starts from CW. If CW has too many routes, the code removes the smallest routes and reinserts their clinics into K remaining slots. This can temporarily overload a route. The search then repairs and improves the plan.

Moves and constant-time cost changes

Relocate moves a clinic to another position or route. **Swap** exchanges two clinics. **2-opt** reverses a segment within one route. **2-opt*** exchanges tails of two routes. The code supports all four. Candidate lists focus the search on nearby clinics, with $k = 20$ in the main experiments.

For a relocate between distinct routes, remove u from edges $(a, u), (u, b)$ and insert it between c and d. The change in cost is

$$\begin{aligned}\Delta = {} & d(a, b) - d(a, u) - d(u, b) \\ & + d(c, u) + d(u, d) - d(c, d).\end{aligned}$$

Only six edge costs are needed, regardless of the number of clinics. Cached route loads give the overload change in constant time too. Applying the chosen move can take longer because route arrays and positions must be updated.

Tabu memory, aspiration and overload

Removed undirected edges become tabu for a random 10–25 iterations. A move reintroducing one is normally forbidden. *Aspiration* allows it if it creates a new best feasible solution.

Moves are ranked by cost plus λ times total overload. If the new plan is overloaded, multiply λ by 1.3; otherwise divide it by 1.3, within fixed bounds. This discourages long periods of overload while letting the search move between clinic assignments. Initially, λ is average depot distance divided by average demand. Only zero-overload plans with at most K routes can become saved answers. TS follows the memory-based search idea in [7].

Algorithm 3 Tabu Search (`tabu.cpp`, moves in `routes.hpp`)

```
Start from CW, reducing to at most K route slots if necessary
Initialise tabu expiry times, penalty and best feasible answer
while time remains do
    Enumerate moves; compute their cost and overload changes
    Reject tabu moves unless aspiration holds
    Choose the best admissible penalised move, even if current cost worsens
    Apply it and mark removed edges tabu
    Adapt the overload penalty
    Save a feasible plan if it improves the best answer
    After stagnation, restart from the best answer with random relocations
Return the best feasible answer
```

Key point. GA searches a population of orders. TS changes an explicit route plan, using short-term memory to discourage cycling back to recently reversed choices.

8 Running-time analysis and stored timing evidence

Let n be clinics, μ GA population, k candidate-list size and L maximum route length. These describe the cost of a step, not how many steps a good answer needs.

Clarke–Wright. There are $O(n^2)$ savings. Sorting costs $O(n^2 \log n)$. At most $n-1$ successful merges each touch $O(n)$ clinics in this implementation. This adds $O(n^2)$ and does not dominate sorting.

GA generation. A generation attempts μ offspring. Per child, crossover, mutation, fast unlimited Split and hashing cost at most $O(n)$ each. Reordering after replacement costs $O(\mu)$. The ordinary generation cost is

$$O(\mu(n+\mu)).$$

If every child needs fleet-limited fast Split, a conservative bound is $O(\mu(Kn+\mu))$. With ordinary Bellman Split, replace the decoding term by $O(nL)$ per child. Keeping μ explicit matters: population 100 is larger than some main benchmarks.

TS iteration. The move generator checks $O(nk)$ candidates. Each uses constant-time delta evaluation and a constant-size tabu check. Applying the selected move costs at most $O(n)$, so an iteration costs $O(nk)$: linear for fixed k, quadratic for $k = n-1$.

Delta evaluation. Fully recomputing a solution costs $O(n)$ per candidate, giving $O(n^2k)$ per iteration instead of $O(nk)$. Both solvers store an $O(n^2)$ distance matrix. GA adds $O(\mu n)$ population memory; TS adds an $O(n^2)$ tabu matrix.

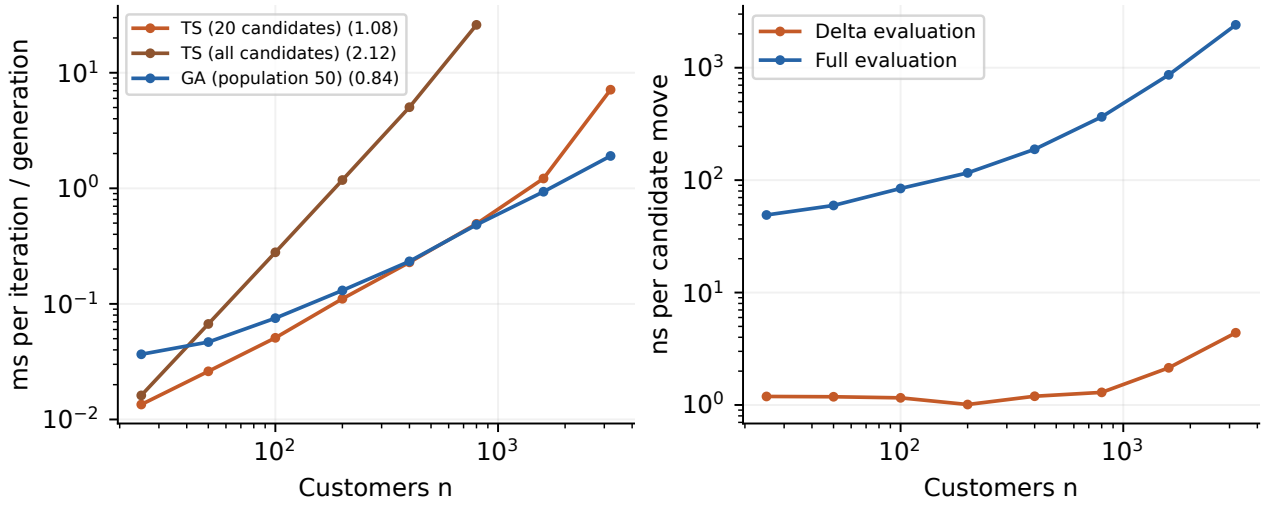


Figure 2: Saved timing data on log–log axes. Left: time per step, with fitted slopes in parentheses. Right: time per candidate for delta and full evaluation.

Measurements. The stored study uses synthetic sizes 25–3200, three repetitions, fixed iteration counts and approximately constant route lengths. Timing two run lengths and subtracting removes initialization. Its GA population is 50, rather than the main comparison's 100. Slopes are 1.08 for fixed-list TS (fit through $n = 1600$), 2.12 for full-list TS and 0.84 for GA. These broadly match predictions 1, 2 and 1 for fixed population and route length. The plotted $n = 3200$ TS point is outside its fit; the sharp increase is consistent with cache and memory effects. GA's fixed overhead matters more for small sizes. A measured slope supports the analysis over those sizes, but is not a proof of asymptotic complexity.

9 Saved results on the required instances

Benchmark gaps use published BKS values. For the team instance the reference is the best value already found in the project, 20,724.21095, rather than a proven optimum. Mean gap is 100(mean cost − reference)/reference.

Instance	Alg.	Best	Mean	SD	Time (s)	Mean gap (%)
A-n32-k5	CW	842.0	–	–	< 0.01	7.40
A-n32-k5	GA	827.0	827.0	0.00	30.00	5.48
A-n32-k5	TS	784.0	784.0	0.00	30.00	0.00
A-n45-k6	CW	997.0	–	–	< 0.01	infeasible
A-n45-k6	GA	944.0	954.9	11.51	30.00	1.15
A-n45-k6	TS	944.0	944.0	0.00	30.00	0.00
A-n60-k9	CW	1,408.0	–	–	< 0.01	3.99
A-n60-k9	GA	1,382.0	1,396.2	6.94	60.00	3.12
A-n60-k9	TS	1,354.0	1,354.0	0.00	60.00	0.00
A-n80-k10	CW	1,840.0	–	–	< 0.01	4.37
A-n80-k10	GA	1,818.0	1,818.0	0.00	60.00	3.12
A-n80-k10	TS	1,763.0	1,763.6	0.97	60.00	0.03
Team (7273)	CW	22,573.3	–	–	< 0.01	infeasible
Team (7273)	GA	21,166.7	21,352.2	114.24	60.00	3.03
Team (7273)	TS	20,724.2	20,725.5	4.05	60.00	0.01

Table 1: GA and TS each have ten runs. SD is sample standard deviation. Time is mean total runtime, not time to best. CW runs once, so mean and SD are omitted. Fleet-infeasible CW costs are diagnostic and receive no gap.

TS reaches the benchmark reference in all ten runs on A-n32-k5, A-n45-k6 and A-n60-k9. On A-n80-k10 it reaches 1763 in six runs, with mean 1763.6. GA has a higher mean on every benchmark. On A-n45-k6, GA sometimes reaches the reference, but less consistently.

On the team instance TS averages about 20,725.5, compared with 21,352.2 for GA. CW needs 19 routes when only 18 are allowed. Each route fits capacity, but the plan is still invalid. These observations concern the implemented methods and settings, not every possible GA or TS.

Tiny instances: exact answers

Instance	n	Exact cost	GA hits	TS hits
tiny-n7-0	7	3180.33	10/10	10/10
tiny-n7-1	7	2753.56	10/10	10/10
tiny-n7-2	7	2873.84	10/10	10/10
tiny-n8-0	8	3069.92	10/10	10/10
tiny-n8-1	8	3535.17	10/10	10/10
tiny-n8-2	8	2987.33	10/10	10/10
tiny-n9-0	9	4212.50	10/10	10/10
tiny-n9-1	9	4359.07	10/10	10/10
tiny-n9-2	9	5307.12	10/10	10/10

Table 2: Stored exact costs and one-second runs reaching each optimum. Cached dynamic-programming and MILP exact values agree.

Each method reaches the exact optimum in all 90 tiny runs. This supports correctness on small cases; it does not guarantee optimality on larger ones. The independent Python checker verifies coverage, loads, fleet size and recomputed cost without using the C++ evaluator.

10 Search progress and the delivery plan

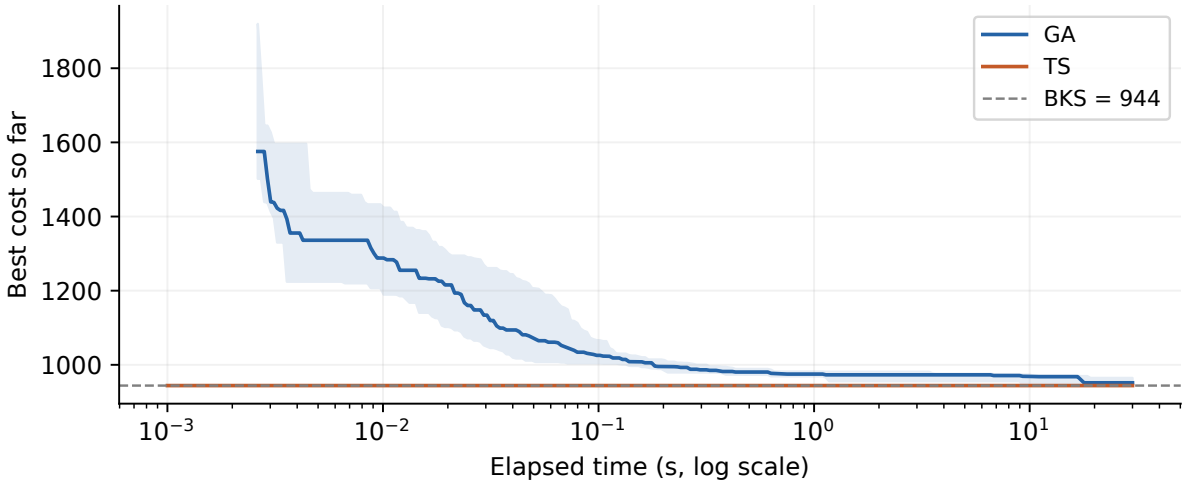


Figure 3: Best cost so far on A-n45-k6. Lines are medians; shading shows the middle 50% of ten runs. Time is logarithmic. Curves start only when all ten runs have a feasible answer.

TS finds a strong solution early here. GA improves more gradually and finishes with different costs across seeds. Best-so-far cost never increases: the current state may worsen, but the algorithm retains its best feasible answer. Early success does not end a run here; the methods continue for the full common time budget.

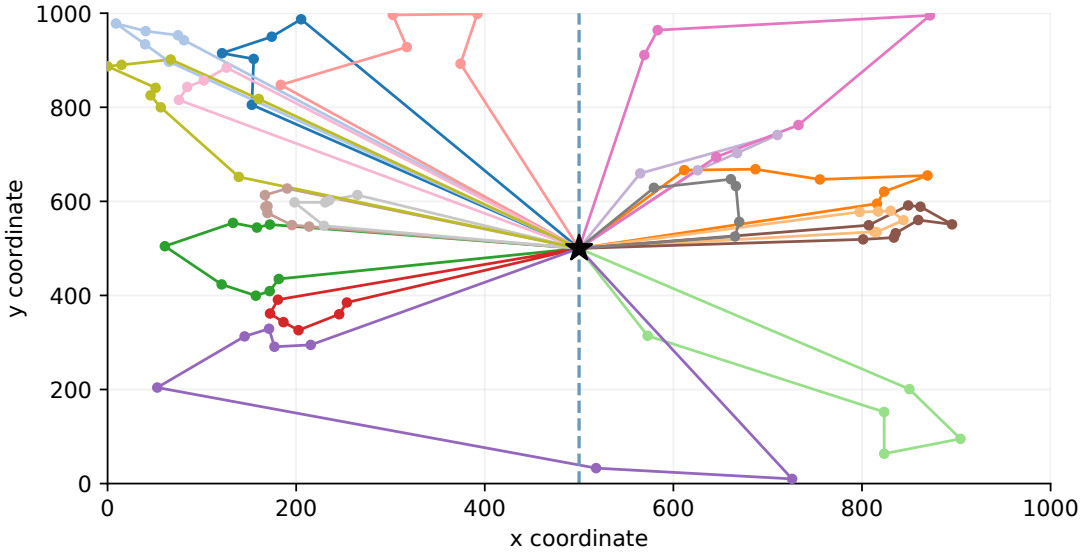


Figure 4: A best saved TS plan for MediDrone-7273, cost 20,724.21095. Each colour is a route. The star is the depot and the dashed line is the synthetic Bosphorus boundary.

This plan uses 17 of the available 18 drones. Ten routes visit Europe; each crosses twice, giving 20 crossing edges. Their penalty is $20 \times 150 = 3000$ cost units. European demand is 938, requiring at least $\lceil 938/100 \rceil = 10$ drones. The plan reaches that simple crossing-route lower bound. This certifies the crossing count, not minimum total distance.

11 Parameter sensitivity and the Bosphorus effect

Tabu tenure

Tenure controls how long removed edges remain forbidden. Very short tenure allows quick reversals and cycling. Very long tenure can block useful moves. The stored study varies tenure on the team instance, fixing the candidate list at 20, with five seeds and ten seconds per run.

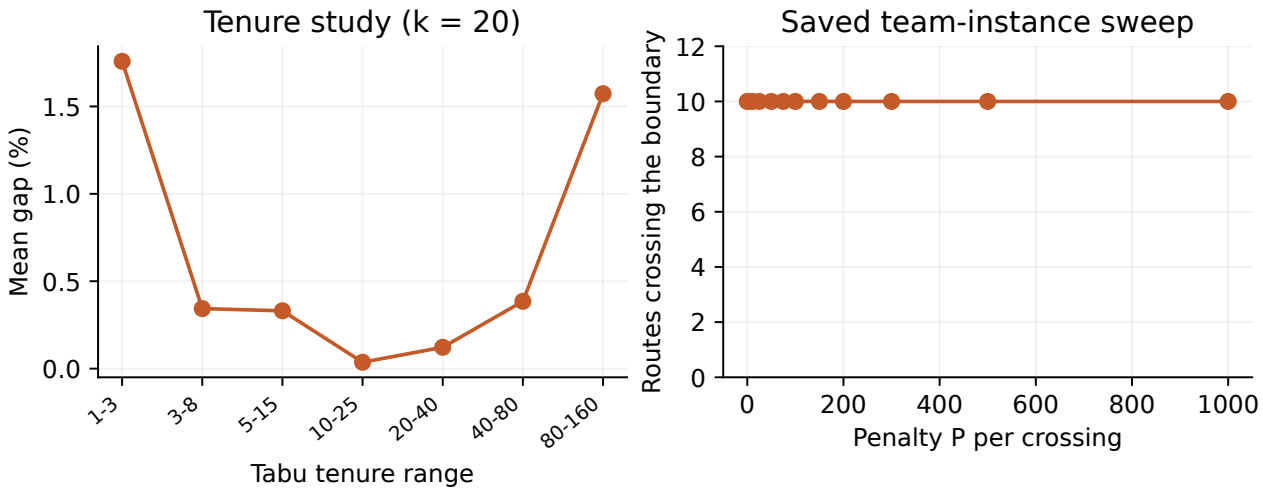


Figure 5: Left: tenure ranges with tuning seeds 101–105. Right: crossing routes in the best saved team-instance plan at each penalty. Both plots reuse existing evidence.

Middle ranges perform better than very small or large ranges. The final range 10–25 lies in this favourable region. This supports a moderate tenure for this instance, rather than a universally optimal setting. Final evaluation uses different seeds, 1–10.

What changing the crossing penalty actually did

The saved sweep covers $P = 0$ to 1000. The best synthetic plans retain ten crossing routes throughout; the cached analysis also records unchanged route structure. Increasing P therefore does not reorganize this instance's best saved routes.

European clinics are grouped far from the depot, and capacity already requires at least ten drones to serve them. A plan reaching that minimum with one out-and-back crossing per such drone cannot reduce crossing cost simply by increasing P. For that unchanged plan, increasing P by one adds 20 to total cost.

The optional real-geography extension does show route changes, but it is a different instance. Its result cannot be presented as a route change on the required synthetic instance.

Why pure GA trails TS here

TS explicitly relocates clinics between routes and improves visit order. Pure GA modifies a giant-tour permutation and relies on Split to create routes. Some useful assignment changes can be harder to obtain through those permutation operators. The saved optional local-search GA performs much better, which is consistent with the value of route-level improvement. This is an interpretation of the observed behaviour, not a proof that the GA representation is inherently inferior.

12 Discussion, limitations and reflection

Theory versus practice

NP-hardness concerns exact answers on every instance. It does not prevent finding good plans quickly on practical inputs. TS often reaches the reference early in these runs, although it does not provide an optimality certificate for the team instance.

Approximation methods offer a proved worst-case relationship to the optimum under stated assumptions. Tour partitioning starts from a TSP tour and cuts it into capacity-feasible routes [8]. The starting tour's quality affects the guarantee. Fleet size needs separate attention: capacity-feasible cuts can still use too many vehicles. Metaheuristics have no general quality guarantee, but can search with the project's constraints and a fixed runtime. In practice, measured quality, repeatability and independent feasibility checks complement theoretical guarantees.

Limitations

Five core instances on one machine do not establish universal TS superiority. Demands are deterministic. The model omits battery range, wind, flight permissions and delivery time windows. It is an algorithm study. The team reference is empirical, and shared hardware affects timing.

Reflection grounded in the project records

The clearest surprise is that the crossing penalty does not reorganize the synthetic instance. Its saved plans already achieve the minimum crossing-route count required by European demand.

Earlier implementation notes record a GA replacement-order bug: after moving a replacement left in a sorted population, the second ordering loop restarted from its old position. This could break the sorted-population invariant and weaken elitism. The recorded fix continues from the updated position in `ga.cpp`. Maintaining this representation invariant matters even when a run's final cost looks plausible.

A useful next step is a controlled comparison with a small local search inside GA, using the existing memetic variant as a starting point. Battery range or time windows could follow after the core algorithms are fully understood.

References

[1] Garey, M. R. and Johnson, D. S. (1979). *Computers and Intractability*. Freeman.

[2] Clarke, G. and Wright, J. W. (1964). Scheduling of vehicles from a central depot to a number of delivery points. *Operations Research*, 12(4), 568–581.

[3] Prins, C. (2004). A simple and effective evolutionary algorithm for the vehicle routing problem. *Computers & Operations Research*, 31(12), 1985–2002.

[4] Vidal, T. (2016). Technical note: Split algorithm in $O(n)$ for the capacitated vehicle routing problem. *Computers & Operations Research*, 69, 40–47.

[5] CVRPLIB. Capacitated Vehicle Routing Problem Library. http://vrp.atd-lab.inf.puc-rio.br/.

[6] ML4CO. *VRPLIBOriDataset*, cached benchmark mirror. https://huggingface.co/datasets/ML4CO/VRPLIBOriDataset.

[7] Glover, F. and Laguna, M. (1997). *Tabu Search*. Kluwer.

[8] Haimovich, M. and Rinnooy Kan, A. H. G. (1985). Bounds and heuristics for capacitated routing problems. *Mathematics of Operations Research*, 10(4), 527–542.

[9] Davis, L. (1985). Applying adaptive algorithms to epistatic domains. *IJCAI*, 162–164.

A Contribution record

The project records one contributor, Faruk Alpay (2267273). Percentages below describe shares among listed students; AI assistance is disclosed separately. The course requests teams of 3–4. This record does not establish approval for an individual submission.

Component	Listed student	Share
Problem definition and theory	Faruk Alpay	100%
Data and instance generation	Faruk Alpay	100%
Solver, exact checks and checker	Faruk Alpay	100%
Experiments and analysis	Faruk Alpay	100%
Report and presentation material	Faruk Alpay	100%

B AI-tool usage statement

The author carried out the project development and research. ChatGPT and OpenAI Codex provided assistance with explanations, debugging, writing, and preparing the report and presentation. The author reviewed the material and remains responsible for the submitted code, results and explanations.

C Reproducing the report without new experiments

From the repository root:

```
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -r requirements.txt
make assets
make report
make check
```

`make assets` runs `analysis/coursework.py`. It reads saved outputs and cached exact/reference values without invoking a solver. Tables go to `report/coursework/`, CSVs to `results/coursework/`, and plots to `figures/coursework/`. `make report` requires LaTeX with `latexmk`, builds this PDF and updates the copy in `deliverables/`.

Evidence	Saved input
Main comparison	`results/raw/main/*.json`
Tiny success counts	`results/raw/tiny/*.json`
Cached exact optima	`results/summary.json` (tiny entries)
Convergence and routes	Main-run traces and routes
Complexity	`results/raw/complexity/complexity.json`
Tenure sensitivity	`results/raw/tune_ts/*.json`
Penalty sweep	`results/psweep_stats.json`

The Python checker uses an independent parser/evaluator. A CW output exceeding K is labelled fleet-infeasible. Timing records carry no route solution and are excluded from route validation.

Code map. `instance.cpp` parses data; `solution.cpp` evaluates routes; `savings.cpp` implements CW; `ga.cpp` and `split.cpp` implement GA/decoding; `tabu.cpp` and `routes.hpp` implement TS/moves. `medidrone/checker.py` is independent of C++. Algorithms follow the cited literature. No existing VRP solver library produces the reported solutions. Comments saying "own implementation" mean project-local code, to be read alongside the AI disclosure.

`docs/REPRODUCIBILITY.md` lists optional experiment commands. They are outside the default build. The earlier research report source is `report/research.tex`, using the original `sections/` and `tables/`; the simplified report uses `coursework/` inputs.